



Mini-Project: Building an Intelligent Agent with LangChain or LangGraph



Submission deadline Feb 19 at 11:59 pm

Presentation deadline Feb 19 at class time. No presentation, no grade.

Project Theme

From Artificial Intelligence a Modern Approach (AIMA) Agents to Real LLM-Based Agents.

In Chapter 2, we studied agents as entities that perceive, maintain state, and act rationally to achieve goals. In this project, you will build a working intelligent agent using modern LLM agent frameworks: LangChain or LangGraph.

You are not expected to know these frameworks yet. Learning them is part of the assignment.

Resources

Learning LangChain and LangGraph, these are the recommended resources to learn to build agents using this framework (maybe the most popular these days).

A nice talk about Agentic AI for Andrew Ng.

<https://www.youtube.com/watch?v=KrRD7r7y7NY>

A simple explanation of agentic AI from another point of view:

https://www.youtube.com/watch?v=15_pppse4fY&t=1s

Overview about langgraph just a high level view:

<https://www.youtube.com/watch?v=CnXdddeZ4tQ>

These videos give you perspective; now you have to take the online courses.



To learn the framework, I strongly recommend following the sequence of online courses available at <https://www.deeplearning.ai/>. I tried skipping some of the courses and found myself quite lost, so I highly encourage you to follow the recommended order.

1. ChatGPT Prompt Engineering for Developer
2. Building Systems with the ChatGPT API
3. LangChain for LLM Application Development
4. LangChain Chat with Your Data
5. Functions, Tools and Agents with LangChain
6. AI Agents in LangGraph

Students who wish to go further may explore advanced topics such as the Model Context Protocol, Retrieval-Augmented Generation (RAG), and related techniques.

Learning Objectives

By the end of this project, you should be able to:

1. Map the AIMA agent model to a real software system (Percepts, State, Actions, Performance Measure).
2. Implement an agent that selects actions (tool use) instead of just generating text.
3. Design and use memory (internal state).
4. Implement decision flow (especially if using LangGraph).
5. Evaluate whether your system behaves like a rational agent, not just a chatbot.

Project Scenario: Student Academic Assistant Agent

You will build an agent that helps a student with academic tasks. The agent must go beyond answering questions — it must decide when to use tools.

Example capabilities include planning study sessions, answering questions based on course documents, checking a draft against a rubric, breaking an assignment into subtasks, and tracking deadlines (simulated).

AIMA Agent Mapping

Percepts: User messages and provided documents.

State: Conversation memory and stored user information.

Actions: Calling tools, asking clarification questions, or providing final answers.

Performance Measure: Usefulness, correctness, proper tool use, and avoiding hallucinations.

You must explicitly explain this mapping in your report.

Technical Requirements

1. Use LangChain or LangGraph (this is the more recent version).
2. Implement at least three tools.



3. Implement both short-term and long-term memory.
4. Demonstrate rational decision behavior (asking clarification, using tools properly, handling missing info).

Example Tools

`search_documents(query)`: Searches provided local course files.
`make_study_plan(goal, time_available)`: Produces a schedule.
`rubric_check(draft_text)`: Evaluates a draft using `rubric.txt`.
`deadline_lookup(task_name)`: Returns due dates from a file.
`summarize_notes(text)`: Produces concise study notes.

The agent must decide when to use tools — not the user.

Memory Requirements

Short-term memory: Conversation summary or history window.

Long-term memory: Stored profile such as course name, deadlines, and user preferences.

Provided Materials

You will receive a folder with sample documents including `syllabus.txt`, `assignment1.txt`, `rubric.txt`, and `faq.txt`. Your agent should use these as its environment knowledge source.

Deliverables

1. Code repository or zip file.
2. Short report (2–4 pages) including environment description (PEAS), AIMA mapping, architecture, tools, memory, discussion of rationality, and limitations.
3. Demo (video or live) showing planning, document lookup, and clarification handling.

Grading Rubric (100 pts)

AIMA mapping: 20

Tool implementation and selection: 20

Memory design and consistency: 15

Decision logic and control flow: 15

Quality and reliability of responses: 20

Documentation and demo: 10

Timeline (2–3 Weeks)

Week 1: Learn LangChain/LangGraph and build basic agent with one tool.

Week 2: Add tools, memory, and decision behavior.

Week 3 (optional): Refinement, testing, and demo preparation.

Key Concept Reminder

Your goal is not to build a chatbot. Your goal is to build a rational agent that uses percepts, maintains state, chooses actions, uses tools, and improves task performance.



Project Advice and Recommendations

1. Start by imagining what your agent looks like

Before writing any code, think at the conceptual level.

What are the inputs (percepts) your agent will receive?

- User questions
- Documents
- Deadlines
- Preferences

What are the outputs (actions) your agent can produce?

- Answers
- Plans
- Tool calls
- Clarifying questions

Try sketching a simple diagram:

User → Agent → Tools / Memory → Response

You may use an LLM to help brainstorm possible features, required tools, project scope, and estimated time. However, the design decisions must be your own.

2. Begin learning Prompt Engineering and LLM Programming

This project is not only about coding; it is about designing agent behavior.

You should understand:

- How prompts influence LLM behavior
- The difference between a single prompt, a chain of prompts, and an agent that decides what to do next

In this context, programming means:

- Designing instructions for the model
- Defining when tools should be used
- Controlling the flow of decisions
- Managing memory and state

You are not just writing functions; you are designing decision logic for an intelligent agent.

3. Choose a Good Development Environment

Use tools that make experimentation easier.

General coding environments:



- VS Code
- Google Colab
- Jupyter Notebooks

These are good for writing Python code, testing tools, and debugging prompts.

Agent-specific tools:

- LangSmith
- LangGraph Studio

These help you understand what the agent is doing internally, see which tools are called, and debug reasoning steps.

4. Build Iteratively

Do not try to build the full system at once.

Recommended order:

1. Make the LLM respond normally
2. Add one tool and test tool calling
3. Add memory
4. Add more tools
5. Improve decision behavior

Small working steps are much better than one large, incomplete system.

5. Test Like an Engineer, Not Just a User

Do not test only ideal prompts. Also test edge cases:

- Missing information
- Confusing instructions
- Conflicting deadlines
- Very long inputs

A good agent asks clarification questions, avoids guessing, and uses tools instead of hallucinating.

6. Keep Scope Realistic

You have only 2–3 weeks. A strong project typically includes:

- One clear domain
- Three to four tools
- Simple memory
- Solid behavior

A small agent that works well is better than a large agent that does not function reliably.



7. Document Your Thinking

Keep notes while building:

- Why you designed tools in a certain way
- Why the agent sometimes fails
- What changes improved performance

This will make your final report much easier to write